今天学习总结

1. 完成 abs 与 RMSNorm 两个手写 npuir kernel，最终 **12 个测试全部通过**（`12 passed in 502.22s`），fp32 RMSNorm 对 torch 参考达到 `max_abs=1.9e-6`。更早的 `11 passed in 473.63s` 是加 `fp16+sqrt_rec` 之前的快照。
2. 踩到 4 个真实的 DSL 坑（`N // block_N` 丢尾、条件分支作用域、`vcast` 不是恒等转换、硬件 `vrsqrt` 只有 ~3e-3 精度），并据此填出第一版旋钮表。
3. 复核性能数据时发现：N=1M 下每次调用被固定开销主导，跨 run 漂 10–30%，多组结论互相矛盾。
4. 拆出模型：耗时 ≈ 固定开销 + 块数 × 每块成本 + 数据搬运；并给出测量与 kernel 两层改法。精度结论全部保留，性能幅度结论降级。
5. **三次复核后的最终口径**：
   - **host 主瓶颈**（profiler + 五点预算闭合）：地板 216–232 μs，两段式（1–256 块平坦；→1024 为 0.04 μs/块；→4096 为 0.26 μs/块），机制在 ACL `rtKernelLaunch` 内。
   - **128 MiB 的 1.070x 不可外推**：同进程 ABBA×10 中位 torch 185.67 vs 我们 200.02 μs，但该尺寸 torch 跨进程自己就漂 167–214 μs（27%），比值会到 0.96–1.09x。
   - **≥256 MiB 张量与 torch 打平**：256 MiB 1.000x（422.57 vs 422.37 μs），512 MiB 1.008x（845.21 vs 838.46 μs），双方 1.27–1.28 TB/s。
   - 8 MiB（3.66x）与 64 MiB（1.21x）的倍数来自块调度和缓存，**不是 HBM kernel 质量**。
6. **产出已经落到仓库里（可复用）**：两个 kernel + 12 个用例 + 24 个探针 + 43 份日志。61 份 `op_summary` CSV，统一放在 `infra-learning/day3/code/`，别人 `cd` 进去就能复跑。
7. **开源贡献（GitHub）**：4 个 skill 随公开仓库 `qingwanruojun/infra-learning` 发布到 `skills/` 目录 —— `ascend-npuir-bench-guardrails`（10 条铁律 + A/B 类失败案例）、`ascend-npu-benchmark`（三时钟分离）、`tilelang-npuir`（DSL 固定套路）、`study-notes-daily`（每日结论/证据分级）。
8. **任务正在部署**：布置的任务已在部署中。

1. 正确性结果

1.1 abs

```
test_abs_divisible        N=4096  block_N=4096   1 block       PASS
test_abs_tail_one         N=4097  block_N=4096   2 blocks,尾1  PASS
test_abs_tail_multi_block N=4100  block_N=1024   5 blocks,尾4  PASS
test_abs_shared           N=4097  alloc_shared   Developer     PASS
test_abs_fp16             N=4097  fp16            Expert        PASS
```

关键点：把示例的 `N // block_N` 换成 `T.ceildiv(N, block_N)` 后，N=4097 的第 2 个 block 只处理 1 个元素，`T.copy` 的动态 slice 正确工作；这正是原示例静默漏掉的路径。

1.2 RMSNorm

（M=N=1024，block_M=8，eps=1e-5，有 weight）【实测，`pytest_all_v2.log`】

dtype	rstd	max_abs	max_rel	判定
fp32	`vsqrt + vrec`	1.907e-6	3.455e-7	远优于 1e-4
fp32	`vrsqrt` + 1 步 Newton	9.441e-5	1.249e-5	通过 1e-4，但余量只有 6%，多 4 条向量指令
fp16	`vrsqrt`	2.344e-2	3.448e-2	在 `atol+rtol·|ref|` 下通过（fp16 输出量化主导）
fp16	`vsqrt + vrec`	9.766e-4 ~ 3.906e-3（有 weight，换输入）；无 weight 另一组 1.953e-3	—	更稳，建议作为 Day 8 基线；测试用 `atol=5e-3` 锁住

另通过：无 weight、M=1023（最后一块 7 行）、`alloc_shared` Developer 模式。合计 **12 个用例**（abs 5 + rmsnorm 7，含新增的 `test_rmsnorm_fp16_sqrt_rec`）。

1.3 rstd 精度探针

`experiments/probe_rstd.py` 对同一批 fp32 行单独测 rstd 的相对误差【探针】：

方法	rstd 相对误差
`T.vrsqrt`	2.8e-3
`T.vsqrt + T.vrec`	~1e-7（与 torch 对拍为 0）
`T.vrsqrt` + 1 次 Newton	1.2e-5

误差定位方法：逐行算 `y / ref`。行内标准差只有 4e-8、整行是同一个比例，说明误差全在 rstd，而不是 reduce 顺序或 cast。探针测的是 rstd 相对误差，1.2 节的 `max_abs` 是端到端 y 的绝对误差，两者量级关系由 `|x|` 放大，排序一致。

结论：fp32 对 1e-4 默认用 `sqrt_rec`（Newton 可作备选，但同形状下 `max_abs` 是 9.441e-5 对 1.907e-6 —— 约差 50 倍，余量只剩 6%，且多 4 条向量指令）；fp16 对 1e-2 可以用 `vrsqrt`，但基线建议用 `sqrt_rec`。

2. 学到的 DSL 事实

2.1 tail-safe 的一维分块

```
num_blocks = T.ceildiv(N, block_N)
...
offset = cid * block_N
tail = T.min(block_N, shape - offset)
T.copy(A[offset : offset + tail], a_buf[0:tail])
T.vabs(a_buf, b_buf)
T.copy(b_buf[0:tail], B[offset : offset + tail])
```

- grid 必须 `ceildiv`；`N // block_N` 在非整除形状下少算尾块，且**不会报错。**

- `tail` 依赖运行时 `shape` 参数，而不是 Python 常量；同一个编译产物能吃不同 N。
- `offset` 越界时 `tail` 为负，`T.copy` 空操作，不会写坏内存。
- `T.vabs` 就是 `npui_abs`，在 Expert 模式直接落 `hivm.hir.vabs`。

2.2 条件分支：只在里面放算子，不要放分配

TVMScript 对 Python bool 的 `if` 会在解析期求值（`visit_if` 里 `isinstance(condition, bool)` 分支），但整段 `if` 包在 `var_table.with_frame()` 里。在 `if` 里 `T.alloc_ub` 得到的名字出了 `if` 就 `Undefined variable`。所以：

```
# 对：分配在外面
x_f32 = alloc(...)
if dtype == accum_dtype:    # 解析期只留一个分支
    T.copy(x_in, x_f32)
else:
    T.vcast(x_in, x_f32, round_mode="rint")
```

2.3 `T.copy` / `T.vcast` 的 dtype 语义

- `T.copy` **不做** dtype 转换：`Check failed: npuirop.src->dtype == npuirop.dst->dtype`。
- `T.vcast(f32→f32, round_mode="rint")` 不是恒等转换，而是**取整到整数**；实测 `-1.0321 → -1`，整个 RMSNorm 输出变成整数。
- `T.vcast(f16→f32, "rint")`、`T.vcast(f32→f16, "rint")` 正常。
- 同 dtype 用 `T.copy`，跨 dtype 用 `T.vcast`，并按 Python bool 分支二选一。

2.4 硬件 `vrsqrt` 是近似

精度数据见 1.3。fp32 达不到 1e-4，必须 Newton 或 `sqrt+reciprocal`；fp16 在 1e-2 容差内可直接用。

2.5 2D tile + 行内 reduce

RMSNorm 选择 `(block_M, N)` 整行进 UB，而不是按 N 切 tile：

```
T.vmul(x_f32, x_f32, xsq)                          # x^2
T.reduce(xsq, sumsq, dims=[1], reduce_mode="sum")  # (block_M,N)->(block_M,1)
T.vmul(sumsq, inv_n, sumsq)                        # 标量在 src2
T.vadd(sumsq, eps, sumsq)
T.vsqrt(sumsq, rstd_tmp); T.vrec(rstd_tmp, rstd)
T.vmul(x_f32, rstd, xsq)                           # 广播 (bM,N)*(bM,1)
T.vmul(xsq, w_f32, xsq)                            # 广播 (bM,N)*(1,N)
```

- `T.reduce` 的硬约束：src/dst 同 rank；恰好一维不同，且 dst 该维为 1。`(block_M, N) → (block_M, 1)` 合法。
- N 全进 UB 后，reduce 不需要跨 tile 累加；这是 1.5 h 内能写完的关键。
- 标量只能放在二元算子的 `src2`；`inv_n` / `eps` 在 `T.Kernel` 内、`T.Scope` 外定义。
- 广播 `(bM,N) × (bM,1)` 和 `(bM,N) × (1,N)` 都可用；后者用于 weight。

2.6 UB 预算

原计划 4 个 buffer ≈128 KiB。实际为了绕开解析器作用域，所有 buffer 无条件分配，fp32 路径：

```
x_in 32K + x_f32 32K + xsq 32K + y_out 32K + w_in 4K + w_f32 4K + tiny
≈ 136 KiB < 192 KiB
```

仍安全，但余量只剩 ~56 KiB。block_M 加到 16 或开多 buffer 会顶到上限；届时把 `x_f32` 别名到 `x_in`，或为 fp32/fp16 各写一个 prim_func。

3. 第一版旋钮表（数据不能作为参考）

abs（N=1048576，alloc_ub，`logs/abs_bench_final.log`）：

实验	配置	耗时	等效带宽
tile 尺寸	block_N=1024 / 2048 / 4096	295 / 290 / 262 μs	28.4 / 28.9 / 32.1 GB/s
非整除	N=1048575 / 1048577	269 / 265 μs	31.2 / 31.7 GB/s
buffer	ub / shared	264 / 342 μs	31.8 / 24.5 GB/s

rmsnorm（M=N=1024, block_M=8，`logs/rmsnorm_bench.log`）：

dtype	weight	rstd	耗时	等效带宽	max_abs
fp32	False	sqrt_rec	318.03 μs	26.4 GB/s	7.153e-07
fp32	False	rsqrt	255.64 μs	32.8 GB/s	1.285e-02
fp32	True	sqrt_rec	235.18 μs	35.7 GB/s	1.907e-06
fp32	True	rsqrt	253.40 μs	33.1 GB/s	2.525e-02
fp16	False	sqrt_rec	273.49 μs	15.3 GB/s	1.953e-03
fp16	False	rsqrt	267.54 μs	15.7 GB/s	9.766e-03
fp16	True	sqrt_rec	268.77 μs	15.6 GB/s	3.906e-03
fp16	True	rsqrt	255.85 μs	16.4 GB/s	2.344e-02

耗时与带宽列不可引用，原因见下节；`max_abs` 一列可信。

4. 测量口径复盘

4.1 现象

同一份代码、同一台机器，连续四轮跑，同一配置（全部是 `Event` 口径，含 host 空隙）：

配置	run1	run2	run3	final
block_N=1024	278 μs	270 μs	270 μs	295 μs
block_N=4096（align 段）	239 μs	257 μs	253 μs	265 μs
N=1048575（尾 1 对齐）	248 μs	248 μs	321 μs	269 μs
memory=shared	245 μs	270 μs	271 μs	342 μs
memory=ub	243 μs	263 μs	258 μs	264 μs

单配置跨 run 漂移约 10%，个别 27–40% 离群；待比较的差异只有 5–15。

4.2 "单轮区间不重叠"也不等于可复现

第二轮稿据 final run 的区间不重叠（1024 的 min 287.5 > 4096 的 max 264.6）得出"4096 快 11.3%"、"shared 慢 29%"。这组观察本身没算错，但有三点让幅度不能引用：

1. **跨 run 合并后区间几乎相接**：1024 的中位在 270–295 μs（最小 263.5）；4096 在 239–265 μs（最大 267.8）。方向稳定，但效应量只有几个到十几 μs。
2. **配置测量顺序固定**：每轮都是 tile 段 1024→2048→4096、buffer 段 ub→shared。设备暖机 / 降频会让"后测的更快 / 更慢"，与参数效应同向，无法区分。交错顺序的脚本尝试过但崩了（`logs/abs_bench_interleaved_aborted.log`，脚本 KeyError），这项控制尚未完成。
3. **单轮自带噪声**：同一配置在同一轮里测两次可以差 1.3%（261.65 vs 265.02 μs），这是单轮比较的分辨率下限。

所以：**方向性结论保留为"下一轮优先验证的假设"，百分比一律作废。**

4.3 归因

（a）块粒度是大头。 在 N=16M 上扫 block_N：

block_N	块数	耗时（16M，Event 口径）
4096	4096	1120 μs
8192	2048	800 μs
16384	1024	274 μs

耗时随块数近似线性：差 846 μs / 差 3072 块 ≈ **0.27 μs per block**，该尺寸下截距接近 0。数据量相同，只改块数就有 4 倍差距，瓶颈是每块固定成本，不是搬运带宽。这一组本身也是升序测量，存在顺序混杂；但 4 倍幅度远超观察到的 10–30% 漂移，判断成立【实测，可信度较高】。profiler 与五点探针进一步把这条成本归到 **host 提交**，不是设备调度成本，见 4.4。

（b）小尺寸被常数项吞掉。 补充扫描 N = 1M / 4M / 16M 做线性拟合。

（c）计时方式放大了常数。 `benchmark()` 用 NPU Event 包住 **10 次 Python 调用的循环**，测的是设备时间轴的总跨度。如果 host 侧每次入队（Python → TileLang wrapper → ACL）比设备执行还慢，设备会在两次 kernel 之间空转，这段空隙会被计入。那 178 μs 是 host 节奏，而非硬件 launch 成本。

（d）和 torch.abs 比要按尺寸分开说。 最终口径：

张量	我们	torch	比值	性质
4 MiB	12.99 μs	3.55	3.66x	缓存内，块调度主导
64 MiB	26.86 μs	22.13	1.21x	缓存内
128 MiB	200.02 μs	185.67	1.070x	**L2 边界，不可外推**
256 MiB	422.57 μs	422.37	**1.000x**	HBM
512 MiB	845.21 μs	838.46	**1.008x**	HBM

所以**"慢 3.7 倍 / 8 倍"不成立（那是 8 MiB 缓存内 + 块调度），"128 MiB 慢 7%"只在单个进程内成立，"≥256 MiB 打平"才是 kernel 质量结论。**

4.4 三尺寸复核：host 是主瓶颈，≥256 MiB 与 torch 打平

> 数据来源：profiler + ABBA 10 轮 + 五点 host 探针，日志已在本机 `notes/day3/day2-day4-20261028/notes/day2/logs/`。所有表格标注口径：`device` 是 profiler 的 kernel 级时长，`Event` 是包多次调用的跨度。

（a）ABBA 对拍，三个尺寸（device 时长）【实测，E3】。 40 块 + 核内串行，block_N=8192，同进程 A-B-B-A 交替 10 轮：

张量	流量	torch 中位	我们中位	比值	torch 极差	我们的 stdev (μs)
128 MiB	256 MiB	185.67 μs	200.02 μs	1.070x	184.2–205.9	2.58
256 MiB	512 MiB	422.37 μs	422.57 μs	**1.000x**	422.2–422.5	0.12
512 MiB	1 GiB	838.46 μs	845.21 μs	**1.008x**	838.2–838.5	0.31

256 / 512 MiB 上双方都稳到 **0.03%**，这是 kernel 质量的结论：**打平**，1.27–1.28 TB/s。128 MiB 那一行**不能外推** —— 同一尺寸 torch 自己跨进程167–214 μs（27%），见（b）下方。

10 轮里有 9 轮比值落在 1.05–1.09，只有 round 0 两侧时间线相交（比值 <1）。v1.3 的 0.93–0.94x 正是取了 **round 0 的冷启动样本**：
torch 从 218.21 μs 掉到 184–186，我们从 204 掉到 197，只有第一轮相交。所以**"持平或略优"不能再写**；但 v1.4/v1.5 把 128 MiB 的 1.070x 写成"HBM 尺度慢 7%"同样不能写 —— 它只是那个进程内的比值。

口径归属：1.070x / 1.000x / 1.008x 都是 **device 时长**；端到端在 4096 块时仍是 host 主导（1084.6 μs，见下表），两者要分开报。

（b）带宽基线（device duration）

张量	流量	torch 耗时	torch 等效带宽	我们耗时	我们的等效带宽
4 MiB	8 MiB	3.55 μs	2.36 TB/s（缓存）	12.99 μs	0.65 TB/s
64 MiB	128 MiB	22.13 μs	6.1 TB/s（缓存）	26.86 μs	5.0 TB/s
128 MiB	256 MiB	185.7–214.4 μs	1.25–1.45 TB/s	200.02 μs	1.34 TB/s
256 MiB	512 MiB	422.37 μs	1.27 TB/s	422.57 μs	1.27 TB/s
512 MiB	1 GiB	838.46 μs	1.28 TB/s	845.21 μs	1.27 TB/s

从 256 MiB 张量起才是稳定的 HBM 主导；128 MiB 以下（含 128）都能被缓存部分服务，带宽数字不代表 HBM 性能。旧表写"从 128 MiB 起已是 HBM 主导"是错的。

（c）host 预算闭合（五点探针，`logs/hostbudget.log`，单位 μs）

块数	wall	cProfile 总	launcher.launch	load_kernel_binary	grid eval
1	231.9	252.4	113.3	53.2	36.6
64	219.0	254.6	107.7	59.0	36.7
256	215.9	248.6	105.2	54.2	37.1
1024	252.4	286.4	135.2	59.4	37.1
4096	1084.6	1122.8	941.9	73.7	47.3

两段式 host 定律：1→256 块平坦在 216–232 μs（常数项 = launcher ~105 μs + `load_kernel_binary` ~55 μs + grid eval ~37 μs 等），256→1024 只有 0.04 μs/块，1024→4096 才涨到 0.26 μs/块。v1.3 的 `max(≈240, 0.27×块数)` 在 1024 块处高估（预测 276，实测 252），缺的正是 `launcher.launch` 那个常数。

launcher 随块数增长这条线查过了，是死路：`tilelang/jit/jit_npu.py:575-665` 只发一次 `rtKernelLaunch(func, blockNum, &args, ...)`；`rtMalloc(workspace_size × blockNum)` 和 `syncBlockLock` 都有 `>0` 守卫，这两个 kernel 都不触发。增量在 ACL 内部，工具链层没有可改的点，直接用 NPUGraph 绕开整条提交路径。

cProfile 总比 wall 高 20–38 μs（walker 自身开销）：**总耗引用 wall，cProfile 只看各项比例。**

固定 NCORE + 核内串行使端到端 1095→299 μs（3.7x），但 device 只快 5.8x（158→27 μs）——收益主要来自少提交。

（d）被推翻的说法。

- torch 166.93 μs（1608 GB/s ≈ 理论峰值）是孤点，另三次 214–217 μs；"我们慢 1.27 倍"不成立。
- "4096 块设备约 800 μs、比 40 块快 30 倍"不成立：device 158 vs 27 μs，只有 5.8x；那 1095 μs 里约 1092 μs 是 host。
- "截距≈0"是两点拟合的假象（见 4.3b）。

- **v1.3 的"0.93–0.94x 持平或略优"不成立**：那是 round 0 暖机样本；
  但后续把它改成"128 MiB 慢 7%"同样不成立 —— 该尺寸不可复现（见 4.4b）。
- **"1M 慢 3.7 倍"与"≥256 MiB 打平"可以同时为真**：前者是缓存内 + 块调度，
  后者才是 HBM 带宽。把它们放进一句话里比较就是口径错误。
- `T.Pipelined` / "单 buffer 只有半链路带宽"对 abs 不适用：计算量≈0，read+write 共享同一条链路，HBM 尺度已经喂满（见 4.5 修正）。

（e）v1.3 的三个【待核】已全部关闭。

1. **计时语义**：所有表都带口径标签（`Event` / `device`）；203 µs 是设备时长，231.9 µs 是 1 块的墙钟，两者不矛盾，但不可混引。
2. **host 预算**：五点探针闭合，缺项就是 `launcher.launch` 的常数（~105 µs）与两段斜率。
3. **样本量**：128 MiB 上 ABBA ×10 轮，9/10 落在 1.05–1.09；256 / 512 MiB 也各 10 轮（σ 0.12 / 0.31 µs）。

剩余未解决项只有两个，都不影响上面的结论：cProfile 与 wall 的 20–38 µs 系统性差（用 wall 做总耗即可）；N=1048575 那个偶发慢点尚未在 profiler 域复测。

4.5 改法

测量层

1. 用 `torch_npu.profiler` / `msprof` 取 **kernel 级 device duration**，不再用 Event 包 Python 循环。
2. 配置顺序随机化或交错（修好 `benchmark` 的 KeyError），避免暖机与参数效应同向。
3. 报告口径：≥256 MiB 张量的绝对耗时；或扫 N 做回归、**报斜率（GB/s）并扣除截距。**
4. 每组 ≥5 次 run 取中位，差异 < 2σ 写"无差别"。
5. 每个配置同时记录 **host 提交 / device duration / end-to-end** 三列，避免把提交成本算进 kernel。

kernel 层

1. 固定核数 + 核内串行，替代"每 tile 一个 block"：

```
NCORE = 40  # 实际值通过接口查，示例里的 20 只是示意
tiles = T.ceildiv(N, block_N)
with T.Kernel(NCORE, is_npu=True) as (cid, _):
    a_buf = T.alloc_ub((block_N,), dtype)
    b_buf = T.alloc_ub((block_N,), dtype)
    for i in T.serial(T.ceildiv(tiles, NCORE)):
        t = cid + i * NCORE
        if t * block_N < shape:
            off = t * block_N
            tail = T.min(block_N, shape - off)
            T.copy(A[off:off + tail], a_buf[0:tail])
            T.vabs(a_buf, b_buf)
            T.copy(b_buf[0:tail], B[off:off + tail])
```

2. 放大 block_N 到 8192，与 40 块 + 核内串行组合；这条路径在 128 MiB 张量上单进程内跑到 1.34 TB/s（该尺寸不可作对拍点，只作旁证）。**【修正】不要把 `T.Pipelined` 当作 abs 的主优化**：计算量≈0，流水不可能超过链路带宽，双缓冲只在低占用时改善延迟隐藏。
3. **NPUGraph 在本 build 上不可用**（v1.5 实测，推翻了"收益最大的一步"）：
   - 默认配置 capture 直接 abort：`Not allow to synchronize captured-stream, stream_id=42`。

launcher 把 `rtKernelLaunch` 包在 `OpCommand::RunOpApi(name, call, true)` 里（`tilelang/jit/jit_npu.py:623`），这个包装会同步流。
    - `TILELANG_ENABLE_TASKQUEUE=0`（新 cache dir）后 capture 成功，但 replay 是 **静默 no-op**：清零 b 后 replay，`count_nonzero(b)=0`；同进程 torch 图 replay 正确（`max_abs_diff=0`），排除测量问题。
    - 结论：裸 `rtKernelLaunch` 没被 ACL 图捕获记录。**要上 graph 必须先改 launcher。** 天花板：device 214 μs 是硬下界，端到端最多降到 ~214 μs，不是 50 μs。
4. ~~查 `launcher.launch` 为何随块数增长~~ —— 已查，死路：单一 `rtKernelLaunch`，`rtMalloc` / `syncBlockLock` 都有 `>0` 守卫，增量在 ACL 内部，工具链层没有可改点。

4.6 收获（具体到"下次怎么做"）

A. 现在能独立做到的 4 件事（都有一天内的实测支撑）

1. **写一个能过形状矩阵的 Vector kernel**：tail-safe 分块（`ceildiv` + 运行时 `tail`），覆盖整除 / 尾 1 / 尾块近满 / 末行少一行 / 非整除 2D，交付 12 个通过的用例（`pytest_all_v2.log`，502.22 s）。
2. **把一次算子耗时拆成三栏**：host wall / device duration / 端到端，并用五点探针闭合 host 预算（launcher ~105 μs + `load_kernel_binary` ~55 μs + grid eval ~37 μs）。

   例：4096 块时 wall 1084.6 μs，device 只有 158 μs —— 能直接指出 1092 μs 在提交路径。
3. **判断一个对拍点能不能用**：先跨 3 个进程跑参考实现，极差 >5% 就弃用。本次在 128 MiB 上测出 torch 自己 167–214 μs，据此把对拍点挪到 256/512 MiB。
4. **给不成立的结论找到反证**：0.93x 来自 round 0、1.070x 来自不稳定的尺寸、"慢 3.7 倍"来自缓存区——三个都能指到具体日志行。

B. 踩过的坑 → 下次的固定动作

#	这次的具体事件（证据）	教训	下次怎么做（可执行）
1	同一配置跨 run 270→295 μs；`shared` 245→342 μs；待比差异只有 5–15%	先证明测量有分辨力，再调参	调参前先跑 **5 轮同配置**，σ/中位 >2% 就先修测量
2	v1.3 用 round 0 得出 0.93–0.94x，后 9/10 轮都是 1.05–1.09	暖机样本会伪装成参数效应	ABBA 交替 + **丢前 1–2 轮** + ≥10 轮取中位
3	128 MiB 上 torch 跨进程 167–214 μs（27%），比值区间 0.96–1.09	对拍点本身要先证明稳定	对拍前同一尺寸跑 **3 个进程**，极差 <5% 才用；否则换 ≥256 MiB
4	4096 块 wall 1084.6 μs vs device 158 μs	提交成本会被当成 kernel 性能，差一个数量级	每个表同时输出 **host / device / e2e 三列**，表头写口径
5	两点拟合出"截距≈0"；五点实测 1–256 块是平台	两点拟合会造出过原点的假象	拟合 ≥3 点，有平台期用 `max` / 分段模型
6	Newton 5.865e-5（M=64）与 1.9e-6（M=1024）被算成"差 30 倍"	跨形状的误差倍数没有意义	精度表**必须带形状列**；不同形状只比"是否达标"，不比倍数
7	NPUGraph 默认 capture abort、关 taskqueue 后 replay 静默 no-op	"收益最大的一步"可能连跑都跑不起来	新优化先写 ≤**30 行可用性探针**（capture→replay→对拍），再排期

C. 还不会的（明确边界，避免误判）：没有 PMU / 缓存命中计数，所以"128 MiB 压在 L2 边界"只是**推断**；没有跨机器数据，所以带宽结论只对这 910B3 成立；
没有改 launcher 的能力，所以 NPUGraph 路线停在 blocked。

5. 与计划的偏差

计划	实际	原因
rstd 用 `T.vrsqrt`	fp32 默认 `vsqrt+vrec`，fp16 基线也改用 `vsqrt+vrec`	硬件 vrsqrt 只有 ~3e-3，达不到 fp32 1e-4
4 个 buffer ≈128 KiB	无条件分配 6 个，≈136 KiB	解析器不允许在 `if` 里分配后被外部使用
对齐 vs 非对齐预期"悬崖"	1M 下三点差异 <2%，未复现稳定悬崖；一次 27% 离群（未在 profiler 域复测，暂按噪声处理）	尾块占比小；需要更细的 N 扫描和 PMU
计划 4.5 h	实际约 60 min（含 12 个测试的全量回归 8 min 22 s）	示例与测试组织可复用；主要时间花在编译（每次 ~45–50 s）和排错
NPUGraph 作为首选优化	本 build 不可用（capture abort / replay 静默 no-op）	launcher 的 `RunOpApi(..., true)` 同步流；裸 `rtKernelLaunch` 不被图捕获

6. 下一步

1. **验证"每波约 1.5 μs"模型**（最便宜、信息量最大）：profiler 跑 N=1M 的40 块串行版，预测 device ~2–4 μs。成立就说明 8 MiB 的 3.66x 是块调度问题，不是 kernel 质量；再用 40 / 80 / 160 / 320 块四点拟合，把推断升到 E4。
2. **对拍点固定到 256/512 MiB**；128 MiB 只作"不可复现"的反例，不再进结论表。同时把 `probe_abba_hbm.py` 的 `NCORE=40` 改成运行时读取。
3. **NPUGraph：blocked。**需要 launcher 把裸 `rtKernelLaunch` 换成可捕获的接口，在此之前不要把它写进"下一步收益"。天花板是 device 214 μs。
4. **在 device 时间域重做旋钮表 v2**：乱序、≥5 run、附 σ；再下 tile / buffer 结论。
5. **默认调度保持 40 块 + 核内串行**（端到端 1095→299 μs，device 158→27 μs）。
6. **block_M 4/8 扫描**（带宽主导区）；block_M=16 先做 aliasing；block_N=16384 同理编不过。
7. **对齐细扫**：N = 1048576 ± {1,2,4,8,16,32}，每点 20+ 轮，确认尾块长度 / 32 B 对齐与慢路径的关系；顺便把 N=1048575 的偶发慢点在 profiler 域复测。
8. **把 reduce 推出 UB**：N > UB 时按 N 切 tile、跨 tile 累加（参考 `examples/norm/rms_norm.py` 的 split-K）。
9. **补测试**：fp16 + `sqrt_rec` 精度锁定；M/N 都不整除的 2D 尾块组合。
10. **减少无条件分配**：fp32/fp16 各写一个 prim_func（或用 `T.macro` 共享 body），把 fp32 的 UB 占用从 136 KiB 降回 ~100 KiB。
11. **D8 / D9 沿用 v2 协议**：以 `sqrt_rec` 版本为手写基线，与 agent 生成结果对比正确性、IR 和端到端耗时。

7. 产出与贡献

7.1 一次性交付物（都在 `infra-learning` 仓库里）

类别	内容	位置	证据 / 状态
算子代码	`abs.py`（tail-safe，ub/shared、fp32/fp16 两条路径）、`rmsnorm.py`（1D/2D tile、weight 开关、三种 rstd）	`day3/code/own_ops/`	12 passed，`logs/pytest_all_v2.log`
测试	5 个 abs + 7 个 rmsnorm，含非整除 / 尾 1 / 末行少一行 / fp16+sqrt_rec	同上（pytest 用例内嵌）	【实测】502.22 s
运行时探针	24 个：host 预算、块数定律、ABBA 对拍、device 阶梯、精度、NPUGraph、静态/动态尾	`day3/code/experiments/`	每个都有对应 `logs/*.log`
实验日志	43 份 `.log` + 61 份 `op_summary` CSV（`logs/profiler_raw/`，含否定记录）	`day3/code/logs/`	【实测】本文所有数字可对到
笔记	v1.6 修订版（本文）、复核修订版、`experiment-log.md`	`day3/note/`、`notes/day3/`	—
结论	device 阶梯 3.66x→1.000x；host 地板 216–232 μs；≥256 MiB 打平；NPUGraph blocked	本文 §4	E3/E4

7.2 GitHub 开源贡献（`skills/`）

仓库：`github.com/qingwanruojun/infra-learning`
→ `skills/`。四个 skill 都是 Claude Code 可直接加载的 `SKILL.md`（frontmatter 里的 `description` 决定什么时候被触发），复制到 `~/.claude/skills/` 或项目的 `.claude/skills/` 即可使用。

Skill	解决什么问题	关键内容
`ascend-npuir-bench-guardrails`	写/评 npuir 算子时反复踩的坑	10 条铁律（ceildiv 尾块、alloc 作用域、copy/cast、vrsqrt 精度、三栏计时、≥256 MiB、ABBA、跨 run、≥3 点拟合、证据分级）+ 7 个 A 类 + 10 个 B 类失败案例
`ascend-npu-benchmark`	"到底比 torch 快还是慢"	三时钟分离、`op_summary` 取 device、ABBA×10、IQR/MAD、尺寸选择
`tilelang-npuir`	新写/改 TileLang NPU kernel	UB 预算、Expert/Developer 模式、动态 shape 尾部、缓存与 launcher 陷阱
`study-notes-daily`	每日结论互相打架	日志/worklog/笔记三层结构、v1.x 版本化、修正对照表

为什么算贡献：它是可执行的工程资产，不是文档。别人（或 agent）在 Day4 之后写新算子时，加载这个 skill 就能自动获得"先过形状矩阵 → 再三栏计时 → ABBA 对拍 → 按证据等级措辞"的默认行为，直接避免本文件里记录的全部返工（例如 128 MiB 当对拍点、round 0 当结论、跨形状比精度）。

7.3 任务部署状态

1.已按指定仓库搭好迁移环境——tilelang-mlir-ascend 框架克隆完成，CANNBOT网关登录成功，8 个 Agent 的 gateway/* 模型全部重映射到实际可用的cannbot/*-Flash，并实跑验证 conductor 能正常调度子 Agent（scaffolder/designer）。

2.环境侧额外做了两项预研：确认 T.verf 只是 7阶泰勒多项式（|x|>1 严重失真，Erf 改用 A&S 7.1.26 公式），以及 BitwiseNot的 dtype 结论（int8/16/32/64 可用 vnot；uint8 需预分配输出；原生 bool会挂死，改用 int8 视图 + vxor）。

3.第一个算子 LeakyReluFwdOp已进入迁移流程：Stage 0 脚手架完成（13 分钟，7 个文件 +.migration_meta.json，GPU 源仓保持只读未改动），当前正在函数级 Stage 1设计阶段，即将产出 DESIGN.md 并进入检视与 NPUIR 开发。

附录 A：复现

```
cd infra-learning/day3/code                                    # 代码/探针/日志都在这里
python -m pytest own_ops/abs.py own_ops/rmsnorm.py -s -q   # 12 个用例，502.22 s（编译占大头）
python own_ops/abs.py                                          # 第一版旋钮表（Event 口径，耗时列作废）
python own_ops/rmsnorm.py                                      # 同上，只有 max_abs 可引用

# 精度探针
python experiments/probe_rstd.py                # 三种 rstd 的 rstd 相对误差
python experiments/probe_copy_cast.py           # copy 跨 dtype 报错 / vcast 取整
```

```
# device 域复核（4.4 的三张表的来源）
python experiments/probe_hostbudget.py          # 五点 host 预算闭合
python experiments/probe_profiler_kernel_time.py  # device 阶梯（先看标签，见 6.1）
python experiments/probe_blockcount_16M.py      # N=16M：4096 块 vs 40 块串行
python experiments/probe_abba_hbm.py 33554432  # 128 MiB 张量（对拍点已作废）
python experiments/probe_abba_hbm.py 67108864  # 256 MiB，E3
python experiments/probe_abba_hbm.py 134217728 # 512 MiB，E3
```

`probe_abba_hbm.py` 的块大小/核数/轮数写死在文件头（`BLOCK, NCORE, ROUNDS = 8192, 40, 10`），
改配置要改源码重编译；`NCORE=40` 是 910B3 的实测值，换卡先查接口。

附录 B：文件

日志根目录：`infra-learning/day3/code/logs/`（已从远端同步进仓库，
本文所有数字都能在这里对到）。

- **pytest**：`pytest_all_v2.log`（12 passed, 502.22 s，当前口径）、`pytest_all.log`（11 passed, 473.63 s，加 `test_rmsnorm_fp16_sqrt_rec` 之前的快照 —— 里面的 Newton `5.865e-05` 是 M=64，不要当同一测试的第二次样本）。
- **第一版旋钮表**（Event 口径）：`abs_bench_final.log`、`abs_bench_run{1,2,3}.log`、`rmsnorm_bench.log`、`rmsnorm_bench_v2.log`。
- **对拍与 host**：`abba_hbm_256MiB.log`、`abba_hbm_512MiB.log`、`abba_hbm.log`（128 MiB，已作废但对得上历史）、`hostbudget.log`、`blockcount_16M.log`、`profiler_kernel_time.log`、`profiler_summary.log`、`clock_state.log`。
- **失败/否定记录**：`abs_bench_interleaved_aborted.log`（脚本 KeyError，与 kernel 无关）、`ladder_bidir_v1_invalid.log`、`npugraph*.log`、`launch_taskqueue_{on,off}.log`。
- **CSV**：61 份 `op_summary_*.csv`，在 `logs/profiler_raw/`（device 时长的原始来源）；13 MB 的 `export_only_prof_dir/` 已删（可从 `profiler_raw/` 重新导出）。

附录 C：修正对照

C.1 v1.4 → v1.5（ABBA ×10 + 五点 host 探针之后）

条目	v1.4 结论	v1.5 实测	处理
与 torch 的差距	0.93–0.94x，持平或略优	ABBA ×10 中位 185.67 vs 200.02 μs = **1.070x**	**推翻**"持平"：0.93x 是 round 0 暖机样本；但 1.070x 本身也在 v1.6 被限定为单进程内（见 C.3）
0.93x 的来源	两轮交替，差 <0.7%	只有 round 0 相交，9/10 轮在 1.05–1.09	**归因修正**
host 定律	`max(≈240 μs, 0.27 μs×块数)`	两段：1–256 平坦 216–232；→1024 0.04/块；→4096 0.26/块	**修正**；原式在 1024 处高估（276 vs 252）
host 预算	未闭合（常数凑不出实测）	五点探针闭合，缺项是 launcher 常数 ~105 μs	**关闭**
计时语义	待核（device / end-to-end 混用）	全部表格加口径标签，203 μs 是 device、231.9 μs 是墙钟	**关闭**
launcher O(块数)	待查，疑为 Python 按块循环	`jit_npu.py:575-665` 单一 `rtKernelLaunch`，增量在 ACL	**死路，撤掉该任务**
样本量	2 轮	10 轮 ABBA	**关闭**

C.2 v1.3 → v1.4（首次 profiler 复核）

条目	融合版原结论	v1.3 实测	处理
与 torch 的差距	N=1M 慢 3.7 倍 / 8 倍	128 MiB 交替对拍 0.93–0.94x	**推翻**"慢 8 倍"，但随后 v1.5 又推翻了"持平"
绝对带宽	32 GB/s，不可引用	≥128 MiB 为 1.25–1.32 TB/s	**限定尺寸后恢复**；N=1M 仍不可引用；v1.6 把稳定区收紧到 ≥256 MiB
0.27 μs/块的归属	设备调度成本	host 提交斜率	**归因修正**
截距≈0	成立	两点拟合假象	**推翻**；v1.5 进一步改为两段式
4096 块设备耗时	约 800 μs，比 40 块快 30 倍（估计）	device 158 vs 27 μs，5.8x	**修正幅度**
`T.Pipelined` / 半带宽	建议改用流水 + 双缓冲	abs 计算量≈0，流水无法超过链路	**撤回**
"快 11.3% / 慢 29%"	幅度作废	host 主导，未在 device 域复测	**维持作废**
host 假设	未验	256 块 host 239.2 vs device 12.99 μs	**验证成立，且比预想更极端**

C.3 v1.5 → v1.6（本修订版；日志落到本机 + 三尺寸 ABBA + 证据分级）

条目	v1.5 写法	v1.6 复核	处理
128 MiB 对拍	中位 1.070x，写成"HBM 尺度慢 7%"	同进程比值成立，但 torch 在同一尺寸跨进程 167–214 μs（27%），比值区间 0.96–1.09x	**撤稿**：该点作废，不再进结论表
≥256 MiB	未测，由 128 MiB 外推	256 MiB **1.000x**（422.57/422.37）、512 MiB **1.008x**（845.21/838.46），σ 0.12/0.31 μs	**新增**：HBM 尺度打平（E3）
128 MiB 是否已出缓存	一处写"≥128 MiB 已完全出缓存"，另一处写"128 MiB 压在 L2 边界"	同尺寸能跑到 1.45 TB/s，高于 512 MiB 的 1.28 TB/s 稳态	判为**推断**（无 PMU），以"256 MiB 起 HBM 主导"为口径
8 MiB 3.66x	倾向读成 kernel 慢	1M 元素 = 4 MiB，缓存内 + 块调度；每波 ~1.5 μs 为**推断**	降级为调度问题，给了证伪实验（§6.1）
64 MiB 1.21x	无带宽口径	128 MiB 流量 / 26.86 μs ≈ 5.0 TB/s，torch ≈ 6.1 TB/s（缓存带宽）	补口径，标明不代表 HBM
计时改法	"报告 ≥64 MiB 的绝对耗时"	64 MiB 仍在缓存	改为 ≥**256 MiB**
NPUGraph	"收益最大的一步"	capture abort / replay 静默 no-op；launcher `RunOpApi(..., true)` 同步流	**blocked**，天花板 device 214 μs
测试数	11 passed / 473.63 s	12 passed / 502.22 s（新增 fp16 sqrt_rec）	更新
Newton 精度	`5.865e-5`（M=64）与 `9.441e-5` 并列，暗示同一测试抖动	两次是 **M=64 vs M=1024**，形状不同	只保留 M=1024 的 9.441e-5，并标注余量 6%
Newton vs sqrt_rec	"差约 30 倍"	9.441e-5 / 1.907e-6 ≈ 50 倍（同形状）	改正倍数
profiler 标签	`tilelang abs, 4096 blocks, N=1M`	4096 是 `block_N`，实际 **256 块**	改标签，不改数字
block_N=1024 的块数	`N=1048576` 写 1025 块	`ceildiv(1048576, 1024) = 1024`	算术笔误改正
fp32 `max_rel`	3.492e-7（来自未归档的另一次 run，输入未固定种子）	`pytest_all_v2.log` 是 1.907e-6 / **3.455e-7**	按日志改正；`max_abs` 两次都是 2^-19，量化主导

条目	v1.5 写法	v1.6 复核	处理
状态文件自相矛盾	`STATUS.md` 同时写"128 MiB 的 1.07x 已作废"和"10 轮收敛到 1.07x"	前者对：作废的是**当作 HBM 证据**，不是数列本身	统一为"单进程内成立、不可外推"
日志位置	"远端机器，尚未同步"	43 份 log + 24 个探针 + 61 份 `op_summary` CSV 已进 `infra-learning/day3/code/`	撤掉"以远端为准"的免责；代码/日志随仓库开源
产出归属	只有笔记	代码、探针、日志、4 个 skill 都进 `infra-learning` 仓库	新增第 7 节"产出与贡献"